

Nix, Channels and the NUR

Rohit Goswami · 2021-07-21 · tools · nix · workflow

A short exploration of multi-user `nix` and interacting with the Nix User Repository without `root`

Background

For multi-user `nix` installations [^fn:1], the `NIX_PATH` variable is empty. Here I briefly go over two approaches to mitigate this, one with `nix-channel` and the other by manual pinning. Note that this post will eventually be superseded for most cases by a better `flake` workflow.

Channels

The idea behind using a channel is essentially that the `tar` at a particular commit / tag will be downloaded and stored, typically at `$HOME/.nix-defexpr/channels`. This is in-fact a symlink typically as seen below.

```
tree -L 2 ~/.nix-defexpr/  
/users/home/rg/.nix-defexpr/  
├─ channels -> /nix/var/nix/profiles/per-user/rg/channels  
└─ channels_root -> /nix/var/nix/profiles/per-user/root/channels
```

This is in keeping with the profile data, since by default in a multi-user setup we have the equivalent of running `nix-env --switch-profile /nix/var/nix/profiles/per-user/rg/profile`.

With that aside out of the way, we can start by setting up baseline channels:

```
# Bad idea, see notes  
# nix-channel --add https://github.com/NixOS/nixpkgs/archive/nixos-21.05.tar.gz nixpkgs  
nix-channel --add https://github.com/NixOS/nixpkgs/archive/nixos-unstable.tar.gz unstable  
nix-channel --add https://github.com/nix-community/NUR/archive/master.tar.gz nur  
nix-channel --update
```

This leads to:

```
tree /nix/var/nix/profiles/per-user/rg/channels  
/nix/var/nix/profiles/per-user/rg/channels  
├─ manifest.nix -> /nix/store/smh16xjbbhcrLzhrbl71nm16dhz7pb5-env-manifest.nix  
├─ nixpkgs -> /nix/store/c1hfsx2v1cy50j7ax3y34gwpzzvrm5-nixpkgs-21.05/nixpkgs  
├─ nur -> /nix/store/za2fqbjl29q6hiip39642hjfa0a59mls-nur/nur  
└─ unstable -> /nix/store/a4bb70ihv8x8zxj0zw4wdafl62n9a92q-unstable/unstable
```

Now we can use these names (`'<nur>'` , `'<unstable>'` and `'<nixpkgs>'`) to write a relatively straightforward `config.nix` as follows:

```
# Uses the channel form of the installation
# Goes in ~/.config/nixpkgs/config.nix
{
  # NUR setup
  pkgs = import <unstable>; # nixos-unstable channel
  packageOverrides = pkgs: {
    nur = import <nur> { # Also defined as a channel
      inherit pkgs;
    };
  };
}
```

The above snippet can be modified to swap out `unstable` for `nixpkgs` as well. It is generally a good idea to define `nixpkgs` anyway to make sure we can do things like `nix-env -iA cowsay -f '<nixpkgs>'`. This allows us to now interact with `nur` packages without any trouble.

```
nix-shell -p nur.repos.mic92.hello-nur
hello
> Hello, NUR!
```

Notes

- We need to define `pkgs` since by default `NIX_PATH` is empty here
- For more on `packageOverrides`, there's [a pill](#) for that
- For a refresher on `profiles`, there's the `nix` manual, [specifically chapter 10](#)
- Defining `nixpkgs` as a channel causes `warning: name collision in input Nix expressions` and it gets skipped
- **The problem with channels** is of course that it is hard to keep control of update which occur when users run `nix-channel --update`

Pinning Packages

To mitigate the impurity introduced by `nix-channel --update` we can opt to instead pin both the `pkgs` and `nur` to a particular hash. The benefit is that we can dispense with any `channel` management and still be guaranteed to have the same behavior across different machines.

The star snippet here is focused around `builtins.fetchTarball`, which takes a `url` and a `sha256` hash. Using this, our `config.nix` now becomes:

```
# A config.nix
{
  pkgs = import (builtins.fetchTarball {
    url = "https://github.com/NixOS/nixpkgs/archive/2588a6c9f1a3279d523e7bd0068da4e12db76ca8.tar.gz";
    sha256 = "0x1q5sf04qk6d05gn4rr2hmlfz17mfkd9fsxwc0kdbhx5yl65zwy";
  });
  packageOverrides = pkgs: {
    nur = import
      (builtins.fetchTarball {
        url = "https://github.com/nix-community/NUR/archive/4a81d63c672ffdbbb999eb2549c0eb0934b3384b.tar.gz";
        sha256 = "09bg0xifdk6rw35w472rk0dj69w08c1ksb3a3qray0ww69495zk";
      })
    {
      inherit pkgs;
    };
  };
}
```

To update things in a controlled manner, use a commit from the repository as the URL, and then:

```
url="https://github.com/nix-community/NUR/archive/3a6a6f4da737da41e27922ce2cfacf68a109ebce.tar.gz"
nix-prefetch-url --unpack $url
```

Which will emit the required hash. With this setup the usage is as before:

```
nix-env -f '<unstable>' -iA nur.repos.mic92.hello-nur
hello
> Hello, NUR!
```

Note that the uninstall is still best done after checking the name with `nix-env -qa`:

```
nix-env -e hello # not hello-nur
```

Bonus

Locale Errors

For some, dropping into `nix-shell` causes `bash: warning: setlocale: LC_ALL: cannot change locale (en_US.UTF-8)`. In this case add the following to the `.bashrc` or its equivalent `rc` file.

```
export LOCALE_ARCHIVE="$(nix-build --no-out-link "<nixpkgs>" -A glibcLocales)/lib/locale/locale-archive"
```

Conclusions

Much of the channel workflow is discussed directly [in this issue](#). The main usage of this is actually to setup and use user defined packages in a more natural way, and to move away from `channels` which typically only update `NIX_PATH` when run as `root`, which in turn [causes much confusion](#). NUR is also `flake` compatible and this will be explored further in a another post.